Setup Guide: EKS Web App (ap-south-1)

Overview

This guide provisions an Amazon EKS cluster in ap-south-1 using Terraform, deploys a containerized Python web app, exposes it via an AWS Application Load Balancer (ALB), and enables autoscaling for both pods and nodes.

Prerequisites

- AWS CLI configured with sufficient permissions
- Terraform ≥ 1.6, kubectl, Helm, Docker
- (Optional) A Route53 hosted zone and ACM certificate for HTTPS

1) Build & Push Image

Replace app.py if desired. Then build and push to ECR:

```
aws ecr create-repository --repository-name saas-web || true
ACC_ID=$(aws sts get-caller-identity --query Account --output text)
docker build -t saas-web:latest -f docker/Dockerfile docker
docker tag saas-web:latest $ACC_ID.dkr.ecr.ap-south-1.amazonaws.com/saas-web:latest
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin
$ACC_ID.dkr.ecr.ap-south-1.amazonaws.com
docker push $ACC_ID.dkr.ecr.ap-south-1.amazonaws.com/saas-web:latest
```

2) Provision Infrastructure (Terraform)

```
cd terraform
terraform init
terraform apply -auto-approve
```

Outputs include cluster name and endpoint.

3) Configure kubectl

```
aws eks update-kubeconfig --region ap-south-1 --name saas-eks
kubectl get nodes
```

4) Deploy Application

```
cd ../k8s
kubectl apply -f namespace.yaml
kubectl -n web apply -f deployment.yaml -f service.yaml -f ingress.yaml -f hpa.yaml

kubectl -n web get deploy,svc,ingress,hpa,pod
```

5) Access the App

Run: kubectl -n web get ingress — copy the ALB DNS and open it in a browser.

6) Optional: HTTPS

Request/validate an ACM certificate in the same region. Add annotations to Ingress:
alb.ingress.kubernetes.io/certificate-arn:
alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS":443}]'

Cost & Performance Tips

• Use SPOT for burst capacity
• Right-size requests/limits for bin packing
• HPA @ 50% CPU, CA enabled
• Consider Graviton (arm64) images for price/perf